

DNN Project Report

Stefano Rosini

Giugno 2025

Indice

1	Introduzione	1
2	Modelli Implementati e Analisi dei pesi	3
2.1	Strategia By-Hand	3
2.2	Default Strategy	4
2.3	Analisi dei Pesi	4
3	Analisi delle Performances	5
3.1	Confronto Modelli SetA1	5
3.2	Confronto modelli SetA2	7
3.3	Confronto Modelli provenienti da insiemi Diversi	8
3.3.1	A1HF vs A2HF	8
3.3.2	A1HT vs A2HT	8
3.3.3	A1DT vs A2DT	8
3.4	A2HF vs modelli del SetA1	8
4	Dettagli Tecnici del progetto	8

1 Introduzione

In questo report discuteremo sulle **performance** ottenute da **sei** Convolutional neural networks. La specifica del progetto chiede di implementare in **python** due insiemi composti da **tre** CNN l'uno. Le CNN che si trovano nello **stesso** insieme condividono la **stessa** architettura:

- Il primo insieme condivide un'architettura denominata **SetA1**, che presenta un layer convoluzionale con cinque kernel di output di dimensione 3×3 , stride=1, padding=1, e dilation=0. Infine ha un layer fully connected.
- Il secondo insieme condivide un'architettura denominata **SetA2**, costituita da due layer convoluzionali: il primo ha un canale di input e cinque kernel di output di dimensione 3×3 , stride=1, padding=1 e dilation=0; il secondo ha cinque canali di input e dieci kernel di output di dimensione 3×3 , stride=1, padding=0 e dilation=0. L'architettura presenta, infine, un layer fully connected.

Entrambe le architetture hanno come funzione di **loss** la **Cross entropy loss** e come **activation function** la **ReLU**. Qui di seguito mostriamo le due implementazioni delle architetture:

```
1 class SetA1(nn.Module):
2     """
3     Class SetA1: Defines the first architecture.
4
5     Architecture Details:
6     - One convolutional layer with 5 output channels, kernel size 3x3, padding=1,
7       ↳ stride=1 and dilation=0.
8     - One max pooling layer (non-trainable).
```

```

8         - One fully connected layer with an input size of 980 and an output size of 10.
9         - Activation function: ReLU.
10        - Total number of trainable parameters:  $5 \times 14 \times 14 \times 10 + 3 \times 3 \times 5 = 9.845$ 
11        """
12
13    def __init__(self, classes=10):
14        super(SetA1, self).__init__()
15
16        torch.manual_seed(0)
17
18        self.conv1 = nn.Conv2d(in_channels=1, out_channels=5, kernel_size=3, padding=1) #
19        ↪ Convolution layer
20        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
21
22        self.fc1 = nn.Linear(5 * 14 * 14, classes)
23        self.relu = nn.ReLU() # Activation Function
24        self.flatten = nn.Flatten()
25
26    def forward(self, x):
27        x = self.pool1(self.relu(self.conv1(x)))
28
29        x = self.flatten(x)
30
31        x = self.fc1(x)
32
33        return x

```

```

1 class SetA2(nn.Module):
2     """
3     Class SetA2: Defines the second type of architecture.
4
5     Architecture Overview:
6     - Two convolutional layers:
7         - The first has 5 output channels, kernel size 3x3, padding=1, stride=1 and
8         ↪ dilation=0.
9         - The second has 10 output channels, 5 input channels, kernel size 3x3,
10        ↪ padding=0, stride=1 and dilation=0.
11     - Two max pooling layers (non-trainable).
12     - One fully connected layer with an input size of 360 and an output size of 10.
13     - Activation function: ReLU.
14     - Total number of trainable parameters:  $5 \times 6 \times 6 \times 10 + 3 \times 3 \times 5 + 10 \times 3 \times 3 = 1800 + 45 + 90$ 
15     ↪ = 1.935
16     """
17
18    def __init__(self, classes=10):
19        super(SetA2, self).__init__()
20        torch.manual_seed(0)
21
22        self.conv1 = nn.Conv2d(in_channels=1, out_channels=5, kernel_size=3, padding=1) #
23        ↪ Convolution layer
24        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
25        self.conv2 = nn.Conv2d(in_channels=5, out_channels=10, kernel_size=3)
26        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
27        self.fc1 = nn.Linear(10 * 6 * 6, classes)
28        self.relu = nn.ReLU()
29        self.flatten = nn.Flatten()
30
31    def forward(self, x):
32        x = self.pool1(self.relu(self.conv1(x)))
33        x = self.pool2(self.relu(self.conv2(x)))

```

```

29     x = self.flatten(x)
30     x = self.fc1(x)
31     return x

```

2 Modelli Implementati e Analisi dei pesi

Come detto prima, questi due insiemi *SetA1*, *SetA2* contengono in totale **sei** architetture, denominate nella seguente maniera:

- per il primo insieme:
 - A1HF;
 - A1HT;
 - A1DT;
- per il secondo insieme:
 - A2HF;
 - A2HT;
 - A2DT;

C'è da notare come i modelli abbiano in comune la parte finale, ovvero **HF,HT,DT**. Questi acronimi identificano le **strategie** di **inizializzazione** e di **addestramento** dei **kernel**:

- HF: Strategia in cui i kernel del primo layer sono inizializzati con dei pesi **impostati manualmente dall'utente** (by-hand strategy) e sono costituiti solo da zeri e da uni, mentre gli altri layer vengono impostati in maniera di **default** (parleremo più in avanti della strategia di default). Tutti i layer, ad eccezione del primo, verranno addestrati. Il primo layer deve mantenere i pesi iniziali fino alla fine dell'addestramento.
- HT: Strategia in cui i kernel del primo layer sono inizializzati con dei pesi **impostati manualmente dall'utente** e sono costituiti solo da zeri e da uni, mentre gli altri layer vengono impostati in maniera di **default** (parleremo più in avanti della strategia di default). Tutti i layer, **incluso** il primo, verranno addestrati.
- DT: Tutti i kernel di tutti i layer sono inizializzati utilizzando la **default strategy** e tutti i layer vengono addestrati.

Di conseguenza, i modelli di insiemi **diversi** che hanno in comune lo **stesso** acronimo avranno la **stessa** strategia di inizializzazione. Chiaramente, i modelli appartenenti allo **stesso insieme** avranno tutti strategie di **inizializzazioni diverse**.

2.1 Strategia By-Hand

La strategia by hand consiste nell'inizializzare i kernel **manualmente** usando solo zeri e uni. I kernel impostati manualmente sono i seguenti:

```

1  kernel1 = torch.tensor([[0, 0, 0],
2                          [1, 1, 1],
3                          [0, 0, 0]], dtype=torch.float32)
4
5  kernel2 = torch.tensor([[0, 1, 0],
6                          [0, 1, 0],
7                          [0, 1, 0]], dtype=torch.float32)
8
9  kernel3 = torch.tensor([[0, 0, 1],
10                         [0, 1, 0],

```

```

11         [1, 0, 0]], dtype=torch.float32)
12
13 kernel4 = torch.tensor([[1, 0, 0],
14                          [0, 1, 0],
15                          [0, 0, 1]], dtype=torch.float32)
16
17
18 kernel5 = torch.tensor([[0, 1, 0],
19                          [1, 1, 1],
20                          [0, 1, 0]], dtype=torch.float32)
21

```

2.2 Default Strategy

La default strategy consiste nell'inizializzare i kernel utilizzando la default initialization.

`nn.Conv2d(...)` utilizza come default initialization la **kaiming uniform initialization**, chiamata anche **he-initialization**: Inizializza i pesi con valori estratti da una distribuzione uniforme centrata in zero. I valori sono scelti casualmente in un insieme $U(-bound, bound)$ dove $bound$ è definito come segue:

$$bound = gain * \frac{1}{\sqrt{fan_in}} \quad (1)$$

$gain$ è un fattore che dipende dal tipo di activation function, mentre fan_in identifica il **numero di connessioni** in ingresso calcolato sul tensore. La **he-initialization** preserva la varianza, evitando l'exploding o il vanishing gradient.

2.3 Analisi dei Pesi

Prima di far partire l'addestramento, è necessario che i pesi dei modelli in questione siano **consistenti**. Per garantire la consistenza è stato implementato un **seed** che garantisce gli stessi risultati ad ogni run del codice. Per verificare se i **pesi iniziali** del primo layer dei modelli appartenenti a due insiemi diversi (Es: primo layer di A1HF e A2HF) e quelli degli altri layer dei modelli appartenenti allo stesso insieme (Es: altri layers di A1HT e A1HF) siano **identici**, si calcola la **norma** della loro differenza. Se i pesi coincidono, il valore della norma risulterà pari a zero:

$$\|W_i - W_j\| = 0 \quad (2)$$

Nel file "*CNN_general.ipynb*" è stato implementato il codice che dimostra l'uguaglianza dei pesi nei diversi scenari dopo l'inizializzazione. Qui di seguito mostro gli screenshot degli output del codice:

```

Distance norm between weights of conv1 layer:
|A1DT-A2DT|: 0.0
|A1HF-A2HF|: 0.0
|A1HT-A2HT|: 0.0
Distance norm between weights of conv1 layer of Same Set:
|A1HF-A1HT|: 0.0
|A2HF-A2HT|: 0.0

```

```

Distance norm between weights of conv2 layer:
|A2HF-A2HT|: 0.0
|A2HF-A2DT|: 0.0
|A2HT-A2DT|: 0.0

```

```

Distance norm between fully connected layers of the 2 sets A1 and A2:
Set A1:
|A1HF - A1DT|: 0.0
|A1HF - A1HT|: 0.0
|A1DT - A1HT|: 0.0
Set A2:
|A2HF - A2DT|: 0.0
|A2HF - A2HT|: 0.0
|A2DT - A2HT|: 0.0

```

Durante l'addestramento, alla fine di ogni epoca vengono salvati i checkpoint. In questo modo possiamo dimostrare anche i Pesì del primo layer convoluzionale dei modelli HF sono rimasti "frezati" per tutto l'addestramento. Qui di seguito mostriamo il codice (che si trova in "*CNN_general.ipynb*") che calcola la differenza della norma dei pesi e il relativo risultato:

```

1
2 checkpoint1_path = ".../SetA*/HF/epoch_1_Model_CNN_HT.pth"
3 checkpoint2_path = ".../SetA*/HF/epoch_50_Model_CNN_HT.pth"
4
5 checkpoint1 = torch.load(checkpoint1_path)
6 checkpoint2 = torch.load(checkpoint2_path)
7
8 model1 = A*HF()
9 model2 = A*HF()
10
11 model1.load_state_dict(checkpoint1['model_state_dict'])
12 model2.load_state_dict(checkpoint2['model_state_dict'])
13
14 if torch.norm(model1.conv1.weight - model2.conv1.weight) == 0:
15     print(f"|conv1_weight_{model1.get_set()}{model1.get_name()}_epoch1 -
16         ↳ conv1_weight_{model2.get_set()}{model2.get_name()}_last_epoch| == 0 --> Freezed
17         ↳ Weights")
18 else:
19     print("Not Freezed Weights")
20
21 # expected results:
22 "|conv1_weight_A1HF_epoch1 - conv1_weight_A1HF_last_epoch| == 0 --> Freezed Weights"
23
24 "|conv1_weight_A2HF_epoch1 - conv1_weight_A2HF_last_epoch| == 0 --> Freezed Weights"

```

3 Analisi delle Performances

Sono stati addestrati tutti i modelli con un numero di epoche pari a 50 e un batch size pari a 128 sul *Fashion Mnist* dataset. Qui di seguito sono riportati tutti i risultati delle performances che hanno restituito i modelli al termine dell'addestramento:

Tabella 1: Performance dei modelli dopo 50 epoche (batch size = 128)

	A1HF	A1HT	A1DT	A2HF	A2HT	A2DT
Accuracy	84.80	85.10	86.39	85.99	86.18	84.90
Avg. Loss	0.42	0.41	0.37	0.39	0.39	0.42

Adesso facciamo i vari confronti delle performances.

3.1 Confronto Modelli SetA1

I modelli che andremo a confrontare in questa sezione saranno i seguenti:

- A1HF;
- A1HT;
- A1DT.

Dalla tabella [1] è possibile notare come, tra tutti i modelli di A1, il modello A1DT abbia l'accuracy e l'average loss **migliore** rispetto alle altre, superando l'86,39% di accuracy e avendo una loss pari a 0.37. Tra i tre, il modello peggiore risulta essere A1HF, con un'accuracy dell'84.80 e una avg loss di 0.42. In conclusione, possiamo dire che l'inizializzazione manuale dei pesi su A1HF e A1HT abbia peggiorato le loro performance, rispetto a A1DT che non li ha inizializzati manualmente. Qui di seguito mostriamo i grafici dell'andamento delle performance di questi modelli:

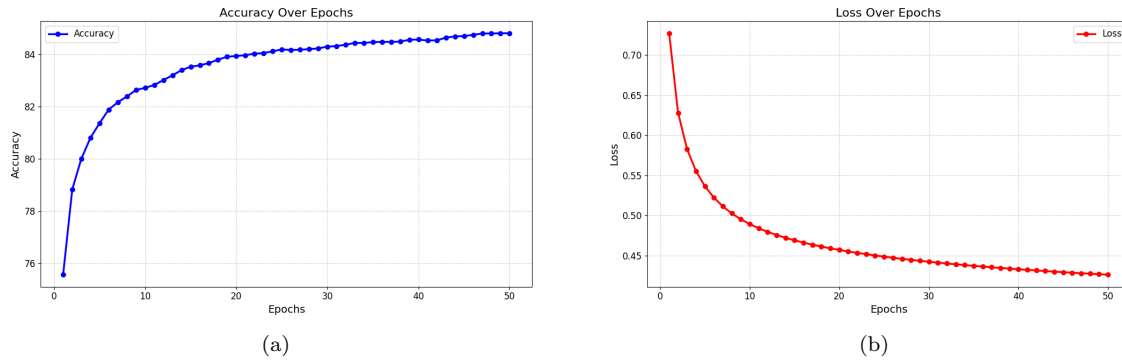


Figura 1: Andamento Accuracy e Loss di A1HF

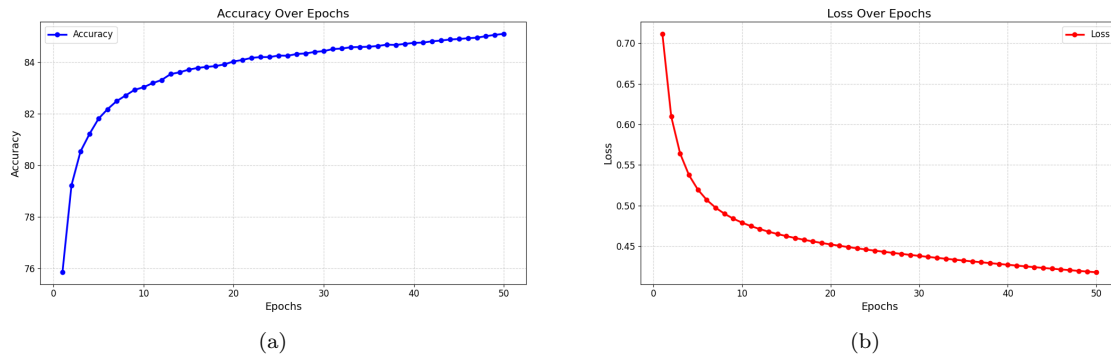


Figura 2: Andamento Accuracy e Loss di A1HT

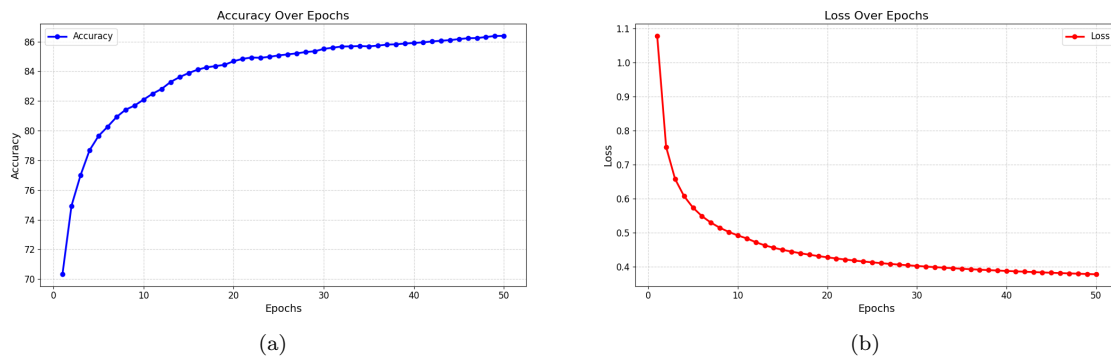


Figura 3: Andamento Accuracy e Loss di A1DT

3.2 Confronto modelli SetA2

Dalla tabella [1] è possibile notare come, tra tutti i modelli di A2, il modello A2HT abbia l'accuracy e l'average loss **migliore** rispetto alle altre, ottenendo l'86,17% di accuracy e avendo una loss pari a 0.39. Il modello A2HF ha la stessa avg loss del modello A2HT. Tra i tre, il modello peggiore di A2 risulta essere A2DT, con un'accuracy del 84.90% e una avg loss pari a 0.42. In conclusione, possiamo dire che l'inizializzazione manuale dei pesi su A2HF e A2HT abbia migliorato le loro performance rispetto a A2DT che non li ha inizializzati manualmente. Qui di seguito mostriamo i grafici dell'andamento delle performance di questi modelli:

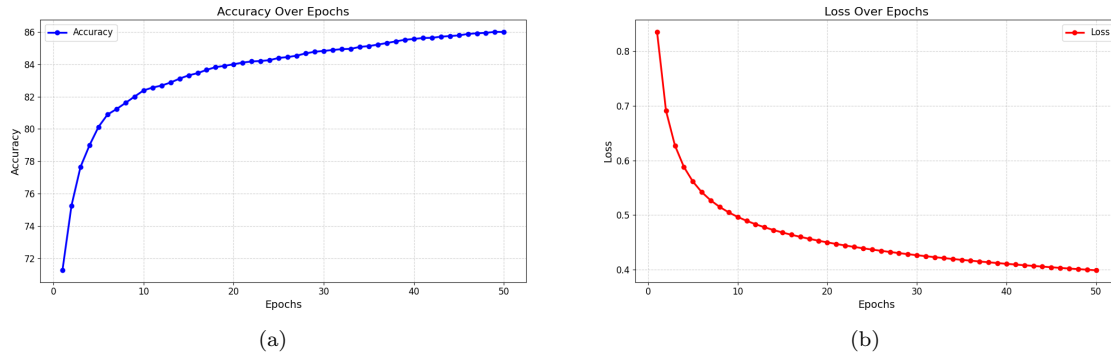


Figura 4: Andamento Accuracy e Loss di A2HF

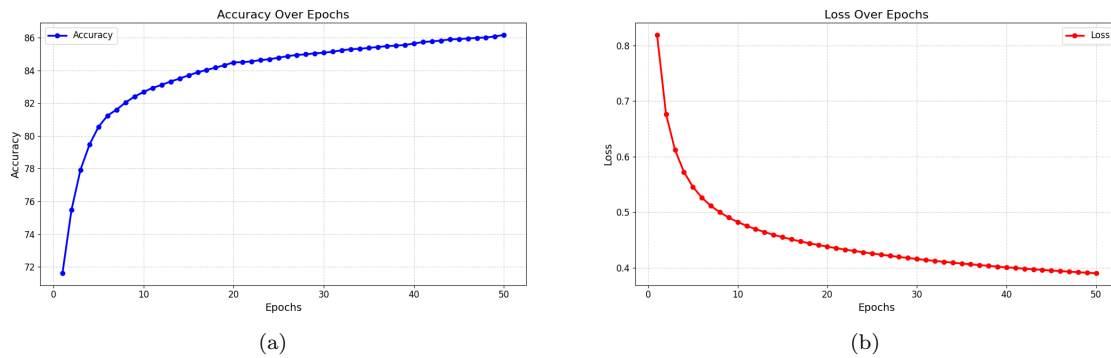


Figura 5: Andamento Accuracy e Loss di A2HT

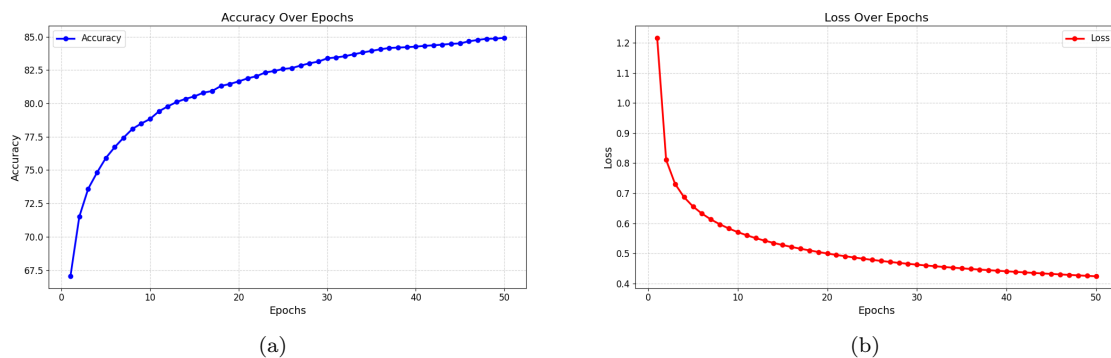


Figura 6: Andamento Accuracy e Loss di A2DT

3.3 Confronto Modelli provenienti da insiemi Diversi

3.3.1 A1HF vs A2HF

Il modello A1HF, implementando l'architettura A1, ha un numero di parametri pari a 9845, mentre A2HF (architettura A2) ne conta 1935. Dalla tabella [1] si osservano le seguenti caratteristiche:

- **Accuracy:** A2HF raggiunge l'85.99 %, contro l'84.80 % di A1HF, ossia un superamento di circa 1,2 punti percentuali.
- **Avg. Loss:** A2HF ottiene una loss media di 0.39, inferiore al valore di 0.42 di A1HF.

In conclusione, A2HF generalizza leggermente meglio di A1HF nonostante sia un modello molto più compatto (a livello di parametri) e complesso.

3.3.2 A1HT vs A2HT

Il modello A1HT, implementando l'architettura A1, ha un numero di parametri pari a 9845, mentre A2HT (architettura A2) ne conta 1935. Dalla tabella [1] si osservano le seguenti caratteristiche:

- **Accuracy:** A2HT raggiunge l'86.18 %, contro l'85.10 % di A1HT, ossia un superamento di circa 1 punto percentuale.
- **Avg. Loss:** A2HT ottiene una loss media di 0.39, inferiore al valore di 0.41 di A1HT.

In conclusione, A2HT generalizza leggermente meglio di A1HT nonostante sia un modello più compatto (a livello di parametri) e complesso.

3.3.3 A1DT vs A2DT

Il modello A1DT, implementando l'architettura A1, ha un numero di parametri pari a 9845, mentre A2DT (architettura A2) ne conta 1935. Dalla tabella [1] si osservano le seguenti caratteristiche:

- **Accuracy:** A1DT raggiunge l'86.39 %, contro l'84.90 % di A2DT, ossia un superamento di circa 1,4 punti percentuale.
- **Avg. Loss:** A1DT ottiene una loss media di 0.37, inferiore al valore di 0.42 di A2DT.

In conclusione, A1DT generalizza meglio di A2DT, ottenendo un'inversione di tendenza rispetto a quello visto in precedenza.

3.4 A2HF vs modelli del SetA1

Dalla tabella [1] si osserva come il modello A2HF sia migliore di A1HF e A1HT sia in termini di performance che in avg loss, ma resta leggermente inferiore ad A1DT. Ciò suggerisce che l'inizializzazione manuale dei pesi **migliora** le prestazioni su modelli un po' più complessi, come A2HF. Tuttavia, senza questa inizializzazione, A1DT ottiene risultati migliori di A2HF.

4 Dettagli Tecnici del progetto

È presente il file *requirements.txt*, che elenca le librerie necessarie per avviare il progetto. Nel file *README.md* sono riportate le istruzioni per l'installazione e l'esecuzione. Il progetto permette l'avvio di 3 modalità:

- Addestramento di un modello alla volta.
- Addestramento di tutti i modelli in sequenza, aventi tutti stesso numero di epoche;
- Caricamento di un checkpoint per far ripartire l'addestramento, qualora si riscontrassero problemi durante l'esecuzione. NB: Dev'essere specificato il path relativo al checkpoint. Quando viene fatta partire la modalità, il folder relativo al checkpoint che prende in considerazione verrà eliminato, per poi essere ricreato automaticamente (per evitare problemi nei salvataggi).

Il progetto prevede, oltre ad una cartella *Checkpoints* in cui vengono salvati quest'ultimi, una cartella di Logs e Plots, in cui rispettivamente vengono salvati i logs dell'addestramento e del testing e i plots relativi agli andamenti delle performances. Qualora non esistessero, il sistema li creerà automaticamente. Ogni volta che verrà avviato il file *main.py*, le cartelle di Logs e Plots verranno eliminate (per evitare problemi durante il salvataggio). I modelli si trovano nella cartella *models* e sono divisi per il *tipo* di insieme a cui appartengono.